

CP Duel Backend: Code-First Function Explanation

Each backend file is explained as: actual code block first, then detailed explanation of what it does, why it exists, side effects, and how to describe it in an interview.

Complete backend flow

Stage	What backend does
Auth	Registers/logs in user, hashes password, returns JWT, verifies JWT on protected routes.
Room	Creates room, stores host as player one, lets second player join, moves status waiting -> ready.
Socket lobby	Players join the same Socket.IO room so lobby updates can be broadcast live.
Start duel	Host calls protected start endpoint; backend validates host/state and selects a Codeforces problem.
Persist start	Room becomes ongoing with currentProblem and startedAt.
Winner detection	Backend polls Codeforces user.status for both handles and verifies Accepted submissions after startedAt.
Finish duel	Room becomes finished, users get wins/losses/rating updates, Socket.IO emits gameResult.

Core interview pitch

The backend is an Express + MongoDB + Socket.IO service. REST endpoints perform authenticated, durable state transitions, MongoDB stores users and rooms, Socket.IO broadcasts real-time state, and Codeforces APIs are used for fair problem selection and server-side winner verification.

src/server.js

Application bootstrap

```
require('dotenv').config();
const express = require('express');
const http = require('http');
const cors = require('cors');
const connectDB = require('./config/db');
const { initSocket } = require('./sockets');

const authRoutes = require('./routes/auth');
const roomRoutes = require('./routes/rooms');
const duelRoutes = require('./routes/duel');

const app = express();
const server = http.createServer(app);

// Connect to MongoDB
connectDB();

// CORS - allow frontend origin
const allowedOrigins = [
  process.env.CLIENT_URL,
  'http://localhost:5173',
  'http://localhost:3000',
];

app.use(cors({
  origin: (origin, callback) => {
    // allow requests with no origin (mobile apps, curl, etc.)
    if (!origin) return callback(null, true);
    if (allowedOrigins.includes(origin)) return callback(null, true);
    return callback(new Error('CORS not allowed for: ' + origin));
  },
  credentials: true,
}));

app.use(express.json());

// Health check
app.get('/health', (req, res) => res.json({ status: 'ok' }));

// Routes
app.use('/api/auth', authRoutes);
app.use('/api/rooms', roomRoutes);
app.use('/api/duel', duelRoutes);

// Init Socket.IO (passes server + app for shared state)
initSocket(server);

const PORT = process.env.PORT || 5000;
server.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Deep explanation

- This file is the runtime entry point. It creates the Express app, wraps it in an HTTP server, connects MongoDB, registers middleware, mounts route modules, attaches Socket.IO, and starts listening on PORT.
- dotenv is loaded first so MONGO_URI, JWT_SECRET, CLIENT_URL, and PORT are available to later modules.
- http.createServer(app) is necessary because Socket.IO needs the raw HTTP server, not only the Express app.
- CORS uses an allow-list. In production, CLIENT_URL should be the deployed frontend URL. Localhost entries support local development.
- express.json() parses JSON request bodies before controllers read req.body.
- The /health route is useful for hosting platforms and quick deployment checks.
- Routes are grouped by domain: auth, rooms, and duel.
- initSocket(server) attaches real-time behavior to the same server process.

Interview answer

Say: server.js wires the backend together. It does not contain business logic; it composes config, routes, middleware, database connection, and sockets.

src/config/db.js

connectDB()

```
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB connected: ${conn.connection.host}`);
  } catch (err) {
    console.error('MongoDB connection error:', err.message);
    process.exit(1);
  }
};
```

Deep explanation

- connectDB is an async startup helper that opens the Mongoose connection using process.env.MONGO_URI.
- If connection succeeds, it logs the MongoDB host. That confirms the backend can reach the database.
- If connection fails, it logs the error and exits with process.exit(1). This is intentional because the app cannot safely serve auth, room, or duel APIs without persistence.
- The function is exported and called once from server.js.

Interview answer

Say: database connection is centralized so server startup has one clear place responsible for MongoDB connectivity.

src/models/User.js

userSchema

```
const userSchema = new mongoose.Schema({
  username: {
    type: String,
    required: true,
    unique: true,
    trim: true,
    minlength: 3,
    maxlength: 20,
  },
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true,
  },
  password: {
    type: String,
    required: true,
    minlength: 6,
  },
  codeforcesHandle: {
    type: String,
    required: true,
    trim: true,
  },
  rating: {
    type: Number,
    default: 1200,
  },
  wins: {
    type: Number,
    default: 0,
  },
  losses: {
    type: Number,
    default: 0,
  },
}, { timestamps: true });

// Hash password before save
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

// Compare password
userSchema.methods.comparePassword = async function (candidatePassword) {
  return bcrypt.compare(candidatePassword, this.password);
};

// Remove password from JSON responses
userSchema.methods.toJSON = function () {
  const obj = this.toObject();
  delete obj.password;
  return obj;
};
```

Deep explanation

- The User schema stores account identity, Codeforces identity, and duel stats.
- username and email are unique, so duplicate accounts are blocked at the database level in addition to controller checks.
- password is required and has minimum length, but the actual stored value becomes a bcrypt hash before save.
- codeforcesHandle connects this app's user to the external Codeforces API.
- rating, wins, and losses are internal game stats. Rating defaults to 1200.
- timestamps automatically adds createdAt and updatedAt.

Interview answer

Say: User combines login identity with the Codeforces handle needed for match verification.

userSchema.pre('save')

```
userSchema.pre('save', async function (next) {  
  if (!this.isModified('password')) return next();  
  const salt = await bcrypt.genSalt(10);  
  this.password = await bcrypt.hash(this.password, salt);  
  next();  
})
```

Deep explanation

- This is Mongoose middleware that runs before saving a user document.
- `this.isModified('password')` prevents re-hashing an already hashed password when other user fields change.
- `bcrypt.genSalt(10)` creates a salt with cost factor 10.
- `bcrypt.hash` replaces the plaintext password with a secure hash before MongoDB stores it.
- `next()` hands control back to Mongoose so the save operation can continue.

Interview answer

Say: password hashing is enforced at the model layer, so controllers do not need to remember to hash manually.

comparePassword()

```
userSchema.methods.comparePassword = async function (candidatePassword) {  
  return bcrypt.compare(candidatePassword, this.password);  
};
```

Deep explanation

- This instance method accepts a plaintext candidate password from login.
- `bcrypt.compare` hashes/checks the candidate against the stored password hash.
- It returns a boolean promise used by `login()`.
- The controller never decrypts a password; it only compares securely.

Interview answer

Say: login verification is delegated to the model through `comparePassword`, keeping auth code readable.

toJSON()

```
userSchema.methods.toJSON = function () {  
  const obj = this.toObject();  
  delete obj.password;  
  return obj;  
};
```

Deep explanation

- This method customizes how a User document becomes JSON.
- It converts the Mongoose document to a plain object, deletes password, and returns the safe object.
- Because controllers return user directly, this prevents password hash leakage in API responses.

Interview answer

Say: `toJSON` is a defensive layer that keeps password hashes out of responses.

src/models/Room.js

roomSchema

```
const roomSchema = new mongoose.Schema({
  roomId: {
    type: String,
    default: () => nanoid(8).toUpperCase(),
    unique: true,
  },
  host: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  players: [
    {
      user: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
      username: String,
      codeforcesHandle: String,
      rating: Number,
    },
  ],
  status: {
    type: String,
    enum: ["waiting", "ready", "ongoing", "finished"],
    default: "waiting",
  },
  currentProblem: {
    contestId: Number,
    index: String,
    name: String,
    rating: Number,
    url: String,
    tags: [String],
  },
  winner: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    default: null,
  },
  winnerUsername: String,
  winnerSubmissionTime: Date,
  ratingDelta: {
    type: Number,
    default: null,
  },
  winnerNewRating: {
    type: Number,
    default: null,
  },
  loserNewRating: {
    type: Number,
    default: null,
  },
  startedAt: Date,
  finishedAt: Date,
}, { timestamps: true })
```

Deep explanation

- Room is the main domain model of the duel. It stores lobby state, players, selected problem, winner, rating outcome, and timestamps.
- roomId is a public 8-character code generated by nanoid and uppercased for sharing.
- host stores the user who created the room and is used to authorize duel start.
- players is an embedded snapshot of each player at match time: user id, username, Codeforces handle, and rating.
- status acts like a state machine: waiting -> ready -> ongoing -> finished.
- currentProblem stores the selected Codeforces problem so refreshes and status calls can recover duel state.
- winner fields and rating fields support the result screen and match history.
- startedAt is critical because polling ignores accepted submissions made before the duel began.

Interview answer

Say: Room is the aggregate root for a match. Most backend flows either create, mutate, or read a Room document.

src/middleware/auth.js

authMiddleware()

```
const authMiddleware = async (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ message: 'No token provided' });
  }

  const token = authHeader.split(' ')[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.userId).select('-password');
    if (!user) return res.status(401).json({ message: 'User not found' });
    req.user = user;
    next();
  } catch (err) {
    return res.status(401).json({ message: 'Invalid or expired token' });
  }
};
```

Deep explanation

- This middleware protects private endpoints.
- It reads the Authorization header and requires the Bearer token format.
- `jwt.verify` checks the token signature and expiration using `JWT_SECRET`.
- The token payload contains `userId`, so the middleware fetches that user from MongoDB.
- `select('-password')` ensures the request user object excludes password hash.
- If verification succeeds, `req.user` becomes available to downstream controllers.
- Any missing, invalid, expired, or stale-user token returns 401.

Interview answer

Say: protected APIs do not trust client-provided user ids; they derive the current user from a verified JWT.

src/routes/auth.js

Auth routes

```
router.post('/register', register);  
router.post('/login', login);  
router.get('/me', authMiddleware, getMe);
```

Deep explanation

- Routes are intentionally thin. They do not contain business logic; they only map HTTP methods and paths to controller functions.
- register and login are public because users need them before having a token.
- me is protected by authMiddleware because it returns the currently authenticated user.

Interview answer

Say: route files keep API mapping separate from controller behavior.

src/routes/rooms.js

Room routes

```
router.post('/create', authMiddleware, createRoom);
router.post('/join', authMiddleware, joinRoom);
router.get('/history', authMiddleware, getHistory); // add this -- must be before /:id
router.get('/:id', authMiddleware, getRoom);
```

Deep explanation

- All room routes are protected because room actions require a logged-in user.
- POST /create creates a room for req.user.
- POST /join joins req.user to a room by roomId.
- GET /history must be defined before GET /:id; otherwise Express would treat 'history' as a dynamic room id.
- GET /:id fetches a room by public roomId.

Interview answer

Say: room routes represent authenticated match/lobby actions.

src/routes/duel.js

Duel routes

```
router.post('/start', authMiddleware, startDuel);  
router.get('/status/:roomId', authMiddleware, getDuelStatus);
```

Deep explanation

- POST /start is protected because only an authenticated host can start a duel.
- GET /status/:roomId is protected so only authenticated users can read duel state.
- The route file delegates all real logic to duelController.

Interview answer

Say: duel routes expose the start/status operations, while the controller handles validation and orchestration.

src/controllers/authController.js

generateToken()

```
const generateToken = (userId) => {  
  return jwt.sign({ userId }, process.env.JWT_SECRET, { expiresIn: '7d' });  
};
```

Deep explanation

- This helper signs a JWT containing the MongoDB user id.
- JWT_SECRET is used to prevent clients from forging tokens.
- expiresIn: '7d' means users remain logged in for seven days unless the frontend removes the token.
- Only userId is stored in the token, keeping the payload small.

Interview answer

Say: JWTs identify the user on later requests without storing server-side sessions.

register()

```
const register = async (req, res) => {  
  try {  
    const { username, email, password, codeforcesHandle } = req.body;  
  
    if (!username || !email || !password || !codeforcesHandle) {  
      return res.status(400).json({ message: 'All fields are required' });  
    }  
  
    // Check existing user  
    const existing = await User.findOne({ $or: [{ email }, { username }] });  
    if (existing) {  
      return res.status(409).json({ message: 'Username or email already taken' });  
    }  
  
    const user = await User.create({ username, email, password, codeforcesHandle });  
    const token = generateToken(user._id);  
  
    res.status(201).json({ token, user });  
  } catch (err) {  
    console.error('Register error:', err);  
    res.status(500).json({ message: 'Server error during registration' });  
  }  
};
```

Deep explanation

- register reads username, email, password, and codeforcesHandle from req.body.
- It validates that all required fields exist before touching the database.
- It checks duplicate email or username using \$or.
- User.create triggers the User model's pre-save hook, so password is hashed automatically.
- After creation, it signs a JWT and returns both token and user.
- Because User.toJSON removes password, the returned user is safe.
- 400 means bad input, 409 means duplicate identity, 500 means unexpected server/database error.

Interview answer

Say: registration creates the user and immediately authenticates them by returning a JWT.

login()

```
const login = async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ message: 'Email and password are required' });
    }

    const user = await User.findOne({ email });
    if (!user) return res.status(401).json({ message: 'Invalid credentials' });

    const isMatch = await user.comparePassword(password);
    if (!isMatch) return res.status(401).json({ message: 'Invalid credentials' });

    const token = generateToken(user._id);
    res.json({ token, user });
  } catch (err) {
    console.error('Login error:', err);
    res.status(500).json({ message: 'Server error during login' });
  }
};
```

Deep explanation

- login reads email and password from req.body.
- It returns 400 if either field is missing.
- It searches by email. If no user exists, it returns the generic 'Invalid credentials' message.
- It calls user.comparePassword(password), which uses bcrypt.compare.
- If password is valid, it signs a new JWT and returns token plus user.
- The same generic 401 message for missing user and wrong password avoids revealing which emails exist.

Interview answer

Say: login verifies credentials securely and gives the client the token needed for protected APIs.

getMe()

```
const getMe = async (req, res) => {
  // req.user is set by authMiddleware
  res.json({ user: req.user });
};
```

Deep explanation

- getMe is tiny because authMiddleware already did the hard work.
- It simply returns req.user, which is the authenticated user document without password.
- This route is useful for restoring frontend auth state after page refresh.

Interview answer

Say: /me proves the token is valid and gives the frontend the current user.

src/controllers/roomController.js

createRoom()

```
const createRoom = async (req, res) => {
  try {
    const user = req.user;

    const room = await Room.create({
      host: user._id,
      players: [{
        user: user._id,
        username: user.username,
        codeforcesHandle: user.codeforcesHandle,
        rating: user.rating,
      }],
      status: 'waiting',
    });

    res.status(201).json({ room });
  } catch (err) {
    console.error('createRoom error:', err);
    res.status(500).json({ message: 'Failed to create room' });
  }
};
```

Deep explanation

- createRoom assumes authMiddleware has already attached req.user.
- It creates a Room with host set to the current user id.
- It also embeds the host as the first player, including username, Codeforces handle, and rating.
- Initial status is waiting because only one player is present.
- The generated roomId comes from the Room schema default.
- The response returns the full room so the frontend can show room code and lobby state.

Interview answer

Say: creating a room persists the lobby and records the host as player one.

joinRoom()

```
const joinRoom = async (req, res) => {
  try {
    const { roomId } = req.body;
    const user = req.user;

    if (!roomId) return res.status(400).json({ message: 'roomId is required' });

    const room = await Room.findOne({ roomId });
    if (!room) return res.status(404).json({ message: 'Room not found' });
    if (room.status !== 'waiting') return res.status(400).json({ message: 'Room is not open for joining' });
    if (room.players.length >= 2) return res.status(400).json({ message: 'Room is full' });

    // Check if user is already in room
    const alreadyIn = room.players.some(p => String(p.user) === String(user._id));
    if (alreadyIn) return res.json({ room }); // idempotent

    room.players.push({
      user: user._id,
      username: user.username,
      codeforcesHandle: user.codeforcesHandle,
      rating: user.rating,
    });
    room.status = 'ready';
    await room.save();

    res.json({ room });
  } catch (err) {
    console.error('joinRoom error:', err);
    res.status(500).json({ message: 'Failed to join room' });
  }
};
```

Deep explanation

- joinRoom reads roomId from req.body and current user from req.user.
- It validates roomId exists, the room exists, the room is still waiting, and the room is not full.

- It checks whether the same user is already in the room. If yes, it returns the room without duplicating the player; this makes the operation idempotent.
- It pushes the second player snapshot into room.players.
- It changes room.status to ready because two players are now available.
- It saves the room and returns it to the frontend.

Interview answer

Say: joinRoom moves the room from waiting to ready while preventing duplicate or over-capacity joins.

getRoom()

```
const getRoom = async (req, res) => {
  try {
    const room = await Room.findOne({ roomId: req.params.id });
    if (!room) return res.status(404).json({ message: 'Room not found' });
    res.json({ room });
  } catch (err) {
    console.error('getRoom error:', err);
    res.status(500).json({ message: 'Failed to get room' });
  }
};
```

Deep explanation

- getRoom fetches by public roomId from req.params.id.
- It returns 404 if no room exists.
- It returns the full room document, which lets frontend pages restore lobby or duel state after refresh.

Interview answer

Say: getRoom is the read endpoint for recovering room state from MongoDB.

getHistory()

```
const getHistory = async (req, res) => {
  try {
    const userId = req.user._id;

    const rooms = await Room.find({
      status: 'finished',
      'players.user': userId,
    })
      .sort({ finishedAt: -1 })
      .limit(20);

    const history = rooms.map((room) => {
      const me = room.players.find(p => String(p.user) === String(userId));
      const opponent = room.players.find(p => String(p.user) !== String(userId));
      const won = String(room.winner) === String(userId);

      return {
        roomId: room.roomId,
        problem: room.currentProblem,
        outcome: won ? 'win' : 'loss',
        opponent: opponent?.username || 'Unknown',
        opponentHandle: opponent?.codeforcesHandle || '',
        finishedAt: room.finishedAt,
        ratingDelta: won ? room.ratingDelta : -(room.ratingDelta || 0),
        myRating: won ? room.winnerNewRating : room.loserNewRating,
      };
    });

    res.json({ history });
  } catch (err) {
    console.error('getHistory error:', err);
    res.status(500).json({ message: 'Failed to fetch history' });
  }
};
```

Deep explanation

- getHistory uses req.user._id to find finished rooms where the user participated.
- The query filters Room documents by status finished and players.user equal to current user.
- Results are sorted newest first and limited to 20.
- For each room, it identifies the current user, opponent, and whether current user won.

- It returns a frontend-friendly history object with roomId, problem, outcome, opponent info, finishedAt, ratingDelta, and myRating.
- ratingDelta is positive for wins and negative for losses.

Interview answer

Say: history is derived from finished Room documents, so there is no separate MatchHistory collection.

src/controllers/duelController.js

setIO()

```
let _io = null;
const setIO = (io) => { _io = io; };
```

Deep explanation

- `_io` stores a module-level Socket.IO server reference.
- `sockets/index.js` calls `setIO(io)` after creating the Socket.IO server.
- `startDuel` can then emit `duelStarted` from inside a REST controller.
- This is a simple dependency injection pattern for sharing `io` without circular runtime setup.

Interview answer

Say: `setIO` bridges REST controllers with real-time socket broadcasts.

startDuel()

```
const startDuel = async (req, res) => {
  try {
    const { roomId } = req.body;
    const user = req.user;

    if (!roomId) return res.status(400).json({ message: 'roomId required' });

    const room = await Room.findOne({ roomId });
    if (!room) return res.status(404).json({ message: 'Room not found' });

    // Only the host can start
    if (String(room.host) !== String(user._id)) {
      return res.status(403).json({ message: 'Only the host can start the duel' });
    }
    if (room.players.length < 2) {
      return res.status(400).json({ message: 'Need 2 players to start' });
    }
    if (room.status === 'ongoing') {
      return res.status(400).json({ message: 'Duel already in progress' });
    }

    // Calculate average rating of the two players
    const avgRating = Math.round(
      room.players.reduce((sum, p) => sum + (p.rating || 1200), 0) / room.players.length
    );

    // Select a Codeforces problem
    const [p1, p2] = room.players;
    const problem = await selectProblem(p1.codeforcesHandle, p2.codeforcesHandle, avgRating);

    const startedAt = new Date();
    room.currentProblem = problem;
    room.status = 'ongoing';
    room.startedAt = startedAt;
    await room.save();

    // Notify all players in socket room
    if (_io) {
      _io.to(roomId).emit('duelStarted', { problem, startedAt });
    }

    // Begin polling for submissions
    const playersForPolling = room.players.map((p) => ({
      userId: p.user,
      username: p.username,
      codeforcesHandle: p.codeforcesHandle,
    }));
    startPolling(roomId, problem, playersForPolling, _io, startedAt);

    res.json({ problem, startedAt });
  } catch (err) {
    console.error('startDuel error:', err);
    res.status(500).json({ message: err.message || 'Failed to start duel' });
  }
};
```

Deep explanation

- `startDuel` is the orchestration function for starting a match.
- It reads `roomId` and current authenticated user.
- It validates `roomId`, room existence, host authorization, minimum two players, and avoids starting if already ongoing.

- It computes average rating from player ratings, defaulting missing ratings to 1200.
- It calls selectProblem with both Codeforces handles and the average rating.
- It records currentProblem, sets status to ongoing, sets startedAt, and saves the room.
- It emits duelStarted to all sockets in that room if Socket.IO is available.
- It builds a compact playersForPolling array and calls startPolling.
- It returns problem and startedAt to the HTTP caller.
- If Codeforces selection fails, the catch block returns a 500 with the error message.

Interview answer

Say: startDuel validates authority and state, selects a fair problem, persists the start, notifies clients, and launches winner detection.

getDuelStatus()

```
const getDuelStatus = async (req, res) => {
  try {
    const room = await Room.findOne({ roomId: req.params.roomId });
    if (!room) return res.status(404).json({ message: 'Room not found' });

    res.json({
      status: room.status,
      currentProblem: room.currentProblem,
      winner: room.winnerUsername,
      winnerSubmissionTime: room.winnerSubmissionTime,
      startedAt: room.startedAt,
      finishedAt: room.finishedAt,
    });
  } catch (err) {
    console.error('getDuelStatus error:', err);
    res.status(500).json({ message: 'Failed to get duel status' });
  }
};
```

Deep explanation

- getDuelStatus reads roomId from params and fetches the room.
- It returns 404 for missing rooms.
- It returns a compact status object rather than the entire room: status, problem, winner, submission time, startedAt, and finishedAt.
- This is useful for frontend refreshes or result pages that need current duel state.

Interview answer

Say: duel status is a read-only endpoint for reconstructing ongoing or finished duel UI.

src/services/codeforcesService.js

fetchProblems()

```
const fetchProblems = async () => {
  const res = await axios.get(`${CF_BASE}/problemset.problems`);
  if (res.data.status !== 'OK') throw new Error('Codeforces API error');
  return res.data.result.problems;
};
```

Deep explanation

- This function calls Codeforces problemset.problems.
- It validates that Codeforces returned status OK.
- It returns only result.problems, which is the list used for filtering.
- If Codeforces is down or returns non-OK, it throws, causing startDuel to fail cleanly.

Interview answer

Say: fetchProblems abstracts the external Codeforces problemset endpoint.

fetchSolvedProblems()

```
const fetchSolvedProblems = async (handle) => {
  const res = await axios.get(`${CF_BASE}/user.status`, {
    params: { handle, from: 1, count: 1000 },
  });
  if (res.data.status !== 'OK') throw new Error(`CF API error for handle: ${handle}`);

  const solved = new Set();
  for (const sub of res.data.result) {
    if (sub.verdict === 'OK' && sub.problem) {
      solved.add(`${sub.problem.contestId}-${sub.problem.index}`);
    }
  }
  return solved;
};
```

Deep explanation

- This function fetches recent submissions for one Codeforces handle.
- It requests up to 1000 submissions, enough to approximate solved history for this project.
- It loops through submissions and only records verdict OK.
- Each solved problem is stored as contestId-index in a Set for O(1) lookup during filtering.
- It throws if Codeforces returns a non-OK response for the handle.

Interview answer

Say: solved sets prevent selecting a problem already solved by either player.

selectProblem()

```
const selectProblem = async (handle1, handle2, avgRating) => {
  const lo = avgRating - 200;
  const hi = avgRating + 200;

  // Banned tags (special or non-standard problems)
  const BANNED_TAGS = ['*special', 'interactive'];

  const [problems, solved1, solved2] = await Promise.all([
    fetchProblems(),
    fetchSolvedProblems(handle1),
    fetchSolvedProblems(handle2),
  ]);

  // Filter suitable problems
  const candidates = problems.filter((p) => {
    const key = `${p.contestId}-${p.index}`;
    const inRatingRange = p.rating >= lo && p.rating <= hi;
    const notSolvedByEither = !solved1.has(key) && !solved2.has(key);
    const noGym = p.contestId < 100000; // Gym contests have IDs >= 100000
    const noIndex = p.contestId !== undefined && p.index !== undefined;
    const hasTags = p.tags && !p.tags.some((t) => BANNED_TAGS.includes(t));
    return inRatingRange && notSolvedByEither && noGym && noIndex && hasTags;
  });

  if (candidates.length === 0) {
    throw new Error('No suitable problem found in rating range');
  }

  const pick = candidates[Math.floor(Math.random() * candidates.length)];
  return {
    contestId: pick.contestId,
    index: pick.index,
    name: pick.name,
    rating: pick.rating,
    tags: pick.tags || [],
    url: `https://codeforces.com/problemset/problem/${pick.contestId}/${pick.index}`,
  };
};
```

Deep explanation

- selectProblem is the core matchmaking/problem-selection algorithm.
- It builds a rating window from avgRating - 200 to avgRating + 200.
- It bans special and interactive problems because they are unsuitable for a standard duel.
- It fetches all problems and both users' solved sets in parallel with Promise.all, reducing start latency.
- It filters candidates by rating range, unsolved-by-both, non-Gym contest id, valid contest/index, and allowed tags.
- If no candidate exists, it throws an explanatory error.
- It randomly picks one candidate and returns a normalized problem object with a direct Codeforces URL.

Interview answer

Say: the algorithm makes the duel fair by using rating range and solved history.

getRecentSubmissions()

```
const getRecentSubmissions = async (handle, count = 10) => {
  const res = await axios.get(`${CF_BASE}/user.status`, {
    params: { handle, from: 1, count },
  });
  if (res.data.status !== 'OK') return [];
  return res.data.result;
};
```

Deep explanation

- This function is used by pollingService during an active duel.
- It fetches recent Codeforces submissions for a handle.
- count defaults to 10 but pollingService passes 15.
- If Codeforces response is not OK, it returns an empty list instead of throwing.
- Returning [] lets polling continue on later intervals without crashing the process.

Interview answer

Say: recent submissions are the raw data used to verify who solved the duel problem.

src/services/pollingService.js

activePolls

```
const activePolls = new Map();
```

Deep explanation

- activePolls maps roomId to intervalId.
- It prevents accidentally starting multiple polling loops for the same room.
- It is in-memory, so it works within one running Node process but will be lost on server restart.

Interview answer

Say: activePolls deduplicates polling, but a production version could use persistent jobs.

startPolling()

```
const startPolling = (roomId, problem, players, io, startedAt) => {
  if (activePolls.has(roomId)) {
    console.log(`Poll already running for room ${roomId}`);
    return;
  }

  const POLL_INTERVAL = 6000; // 6 seconds

  const intervalId = setInterval(async () => {
    try {
      // Check each player's recent submissions in parallel
      const checks = players.map(async (player) => {
        const subs = await getRecentSubmissions(player.codeforcesHandle, 15);
        for (const sub of subs) {
          // Only look at submissions after the duel started
          const subTime = new Date(sub.creationTimeSeconds * 1000);
          if (subTime < startedAt) continue;

          if (
            sub.verdict === "OK" &&
            sub.problem.contestId === problem.contestId &&
            sub.problem.index === problem.index
          ) {
            return { player, submission: sub, subTime };
          }
        }
      });
      return null;
    } catch (err) {
      console.error(`Polling error for room ${roomId}:`, err.message);
    }
  }, POLL_INTERVAL);

  activePolls.set(roomId, intervalId);
  console.log(`Started polling for room ${roomId}`);
};
```

Deep explanation

- startPolling begins winner detection for one room.
- It first checks activePolls; if a poll already exists for the room, it exits.
- It creates a setInterval that runs every 6000 ms.
- For each player, it fetches recent submissions in parallel.
- It converts Codeforces creationTimeSeconds to a Date.
- It ignores submissions made before startedAt, preventing old accepted submissions from winning.
- It checks for verdict OK and exact match on contestId and index.
- Each player check returns either a winner object or null.
- Promise.all waits for both players' checks.
- The first non-null result becomes winner, then polling stops and handleWinner updates state.

- Errors inside polling are logged but do not crash the backend.

Interview answer

Say: winner detection is server-side polling against Codeforces, not a client claim.

stopPolling()

```
const stopPolling = (roomId) => {  
  if (activePolls.has(roomId)) {  
    clearInterval(activePolls.get(roomId));  
    activePolls.delete(roomId);  
    console.log(`Stopped polling for room ${roomId}`);  
  }  
};
```

Deep explanation

- stopPolling checks whether a room has an active interval.
- clearInterval stops future Codeforces checks.
- The room is deleted from activePolls to free memory and allow future cleanup.
- It is called when a winner is found.

Interview answer

Say: stopPolling prevents unnecessary Codeforces API calls after the duel is over.

calcEloDelta()

```
const calcEloDelta = (winnerRating, loserRating) => {  
  const K = 32;  
  const expected = 1 / (1 + Math.pow(10, (loserRating - winnerRating) / 400));  
  return Math.round(K * (1 - expected));  
};
```

Deep explanation

- calcEloDelta computes an Elo-style rating change.
- K = 32 controls maximum rating movement.
- expected is the winner's expected score based on rating difference.
- If the winner was expected to win, delta is smaller; if the winner was lower rated, delta is larger.
- Math.round returns an integer rating change.

Interview answer

Say: rating updates are simple Elo-inspired scoring, not Codeforces official rating.

handleWinner()

```
const handleWinner = async (roomId, { player, subTime }, io) => {
  try {
    const room = await Room.findOne({ roomId, status: "ongoing" });
    if (!room) return;

    // Identify winner and loser players from room
    const winnerPlayer = room.players.find(
      (p) => String(p.user) === String(player.userId)
    );
    const loserPlayer = room.players.find(
      (p) => String(p.user) !== String(player.userId)
    );

    const winnerRating = winnerPlayer?.rating || 1200;
    const loserRating = loserPlayer?.rating || 1200;
    const delta = calcEloDelta(winnerRating, loserRating);

    // Update room doc
    // Update room doc
    await Room.findOneAndUpdate(
      { roomId, status: "ongoing" },
      {
        status: "finished",
        winner: player.userId,
        winnerUsername: player.username,
        winnerSubmissionTime: subTime,
        finishedAt: new Date(),
        ratingDelta: delta, // add this
        winnerNewRating: winnerRating + delta, // add this
        loserNewRating: Math.max(0, loserRating - delta), // add this
      }
    );

    // Update winner: wins++, rating += delta
    await User.findByIdAndUpdate(player.userId, {
      $inc: { wins: 1, rating: delta },
    });

    // Update loser: losses++, rating -= delta (floor at 0)
    if (loserPlayer) {
      const newLoserRating = Math.max(0, loserRating - delta);
      await User.findByIdAndUpdate(loserPlayer.user, {
        $inc: { losses: 1 },
        $set: { rating: newLoserRating },
      });
    }

    // Emit result with delta info
    io.to(roomId).emit("gameResult", {
      winner: player.username,
      winnerHandle: player.codeforcesHandle,
      submissionTime: subTime,
      problem: room.currentProblem,
      ratingDelta: delta, // winner gains this
      winnerNewRating: winnerRating + delta,
      loserNewRating: Math.max(0, loserRating - delta),
    });

    console.log(`Winner: ${player.username} | Delta +${delta}`);
  } catch (err) {
    console.error("handleWinner error:", err.message);
  }
};
```

Deep explanation

- handleWinner finalizes the duel after polling finds an accepted submission.
- It loads the room only if status is still ongoing; this avoids updating an already finished room.
- It identifies winnerPlayer and loserPlayer from room.players using user ids.
- It reads both players' ratings and calculates delta.
- It updates the Room to finished and stores winner, winnerSubmissionTime, finishedAt, ratingDelta, and new ratings.
- It increments winner wins and rating.
- It increments loser losses and sets loser rating with a floor of zero.
- It emits gameResult to the Socket.IO room with winner, problem, submission time, and rating details.
- All errors are caught and logged.

Interview answer

Say: handleWinner is the final transaction-like step, though production should use MongoDB transactions for stronger consistency.

src/sockets/index.js

socketRoomMap

```
const socketRoomMap = new Map();
```

Deep explanation

- socketRoomMap tracks which room a socket joined.
- It is used during disconnect to clean the map entry.
- The map is only real-time connection state, not persistent match state.

Interview answer

Say: sockets track live connections, while MongoDB tracks durable room state.

initSocket()

```
const initSocket = (server) => {
  const io = new Server(server, {
    cors: {
      origin: [process.env.CLIENT_URL, 'http://localhost:5173', 'http://localhost:3000'],
      methods: ['GET', 'POST'],
      credentials: true,
    },
  },
  // Important for Render (WebSocket behind proxy)
  transports: ['websocket', 'polling'],
  });

  // Give the duelController a reference to io
  setIO(io);

  io.on('connection', (socket) => {
    console.log(`Socket connected: ${socket.id}`);

    /**
     * createRoom event
     * Client emits after creating room via REST API
     * { roomId, username }
     */
    socket.on('createRoom', ({ roomId, username }) => {
      socket.join(roomId);
      socketRoomMap.set(socket.id, roomId);
      console.log(`${username} created and joined socket room ${roomId}`);
    });

    /**
     * joinRoom event
     * { roomId, username, players } -- players array from updated Room doc
     */
    socket.on('joinRoom', ({ roomId, username, players }) => {
      socket.join(roomId);
      socketRoomMap.set(socket.id, roomId);
      console.log(`${username} joined socket room ${roomId}`);

      // Notify everyone in the room that a new player joined
      io.to(roomId).emit('playerJoined', { players, roomId });
    });

    /**
     * startGame event
     * Host triggers this; actual logic runs through REST /api/duel/start
     * This event is informational -- confirms the start request was sent
     */
    socket.on('startGame', ({ roomId }) => {
      io.to(roomId).emit('gameStarting', { roomId });
    });

    /**
     * leaveRoom event
     * Player explicitly leaves
     */
    socket.on('leaveRoom', ({ roomId, username }) => {
      socket.leave(roomId);
      socketRoomMap.delete(socket.id);
      io.to(roomId).emit('playerLeft', { username });
    });

    /**
     * On disconnect -- clean up room membership
     */
    socket.on('disconnect', () => {
      const roomId = socketRoomMap.get(socket.id);
      if (roomId) {
        socketRoomMap.delete(socket.id);
        // If nobody left in socket room, room is effectively empty
        // Polling will eventually timeout or the room stays in DB
      }
      console.log(`Socket disconnected: ${socket.id}`);
    });
  });

  return io;
};
```

Deep explanation

- initSocket attaches a Socket.IO server to the HTTP server.
- Its CORS config mirrors the Express CORS origins and supports credentials.
- transports includes websocket and polling, useful for hosted environments like Render.
- setIO(io) gives duelController access to emit duelStarted from REST logic.
- On connection, the server registers event handlers for room creation, joining, starting, leaving, and disconnect.

- createRoom joins the socket to a room after REST room creation.
- joinRoom joins the socket and broadcasts playerJoined.
- startGame only broadcasts gameStarting; actual duel start still happens through REST /api/duel/start.
- leaveRoom removes socket membership and broadcasts playerLeft.
- disconnect removes the socket id from socketRoomMap.

Interview answer

Say: Socket.IO is used for live synchronization, but it does not replace authenticated REST state changes.